

CSE 472 Assignment 2: Decision Trees, Random Forests, and Extra Trees

Submitted by
Zia Ul Hassan Abdullah
2005037

January 25, 2026

Contents

1	Introduction	3
2	Algorithmic Details	3
2.1	Decision Tree	3
2.2	Random Forest	4
2.3	Extremely Randomized Trees	4
3	Experimental Setup	5
3.1	Datasets	5
3.2	Hyperparameters	5
3.2.1	Decision Tree	5
3.2.2	Random Forest	5
3.2.3	Extra Trees	5
3.3	Reproducibility	5
3.4	Evaluation Metrics	6
4	Results and Analysis	6
4.1	Custom vs Scikit-learn Comparison	6
4.1.1	Iris Dataset	6
4.1.2	Wine Dataset	6
4.2	Hyperparameter Analysis	6
4.2.1	Decision Tree	7
4.2.2	Random Forest	7
4.2.3	Extra Trees	7
4.3	Decision Trees vs Ensemble Methods	8
4.4	Confusion Matrix Analysis	9
5	Conclusion	9

1 Introduction

Decision trees are intuitive machine learning models for classification and regression tasks. However, single decision trees often suffer from high variance and overfitting. Ensemble methods such as Random Forests and Extremely Randomized Trees mitigate these issues by combining multiple randomized trees.

In this assignment, we implemented Decision Trees, Random Forests, and Extra Trees from scratch, analyzed their performance empirically, and compared them with optimized implementations from scikit-learn.

The objectives of this work are:

- Understanding the working principles of decision trees
- Implementing ensemble learning techniques
- Analyzing bias–variance tradeoffs
- Comparing custom models with industrial-grade libraries

2 Algorithmic Details

2.1 Decision Tree

Our implementation follows the CART (Classification and Regression Tree) framework. Each internal node performs a binary split of the form:

$$x_j \leq t \quad \text{vs} \quad x_j > t \tag{1}$$

where x_j is the selected feature and t is a threshold.

For continuous features, candidate thresholds are computed as midpoints between sorted unique values.

At each node, the best split is selected by maximizing Information Gain:

$$IG = I(\text{parent}) - \sum_{k \in \{L, R\}} \frac{N_k}{N} I(k) \tag{2}$$

where $I(\cdot)$ denotes impurity.

Two impurity measures are supported:

Gini Index

$$\text{Gini} = 1 - \sum_c p_c^2 \tag{3}$$

Entropy

$$H = - \sum_c p_c \log_2 p_c \tag{4}$$

Tree growth stops when:

- All samples belong to one class
- Minimum samples per split is violated

- Maximum depth is reached

Leaf nodes store empirical class distributions, enabling probability prediction.

2.2 Random Forest

Random Forest is an ensemble of M decision trees.

Each tree is trained using:

- Bootstrap sampling
- Random feature subspace selection

For each tree:

$$D_i \sim \text{Bootstrap}(D) \quad (5)$$

where D is the original dataset.

At each split, only a subset of features is considered:

$$m = \sqrt{p} \quad (6)$$

where p is the total number of features.

Final predictions are obtained by averaging class probabilities:

$$P(y = c|x) = \frac{1}{M} \sum_{i=1}^M P_i(y = c|x) \quad (7)$$

2.3 Extremely Randomized Trees

Extra Trees introduce additional randomness compared to Random Forests.

Differences include:

- No bootstrap sampling (by default)
- Randomly selected thresholds
- Random feature subsets

Instead of searching for optimal thresholds, thresholds are sampled uniformly:

$$t \sim U(\min(x_j), \max(x_j)) \quad (8)$$

This increases bias slightly but significantly reduces variance and training time.

Predictions are aggregated in the same manner as Random Forests.

3 Experimental Setup

3.1 Datasets

Experiments were conducted on:

- Iris Dataset
- Wine Dataset

Both datasets were obtained from `sklearn.datasets` and split into 80% training and 20% testing sets using stratified sampling.

3.2 Hyperparameters

The following hyperparameters were used.

3.2.1 Decision Tree

- `max_depth`: None
- `min_samples_split`: 2
- `criterion`: Entropy

3.2.2 Random Forest

- `n_estimators`: 100
- `max_depth`: None
- `min_samples_split`: 2
- `max_features`: sqrt
- `bootstrap`: True

3.2.3 Extra Trees

- `n_estimators`: 100
- `max_depth`: None
- `min_samples_split`: 2
- `max_features`: sqrt
- `bootstrap`: False

3.3 Reproducibility

All experiments were conducted using a fixed random seed:

$$\text{Seed} = 42 \tag{9}$$

This ensures reproducibility.

3.4 Evaluation Metrics

Performance was measured using:

- Accuracy
- Macro F1-score
- AUROC (One-vs-Rest)

4 Results and Analysis

4.1 Custom vs Scikit-learn Comparison

Tables below summarize performance.

4.1.1 Iris Dataset

Model	Impl	Dataset	Accuracy	F1	AUROC
DT	Custom	Iris	0.933	0.933	0.950
DT	Sklearn	Iris	0.933	0.933	0.950
RF	Custom	Iris	0.933	0.933	0.993
RF	Sklearn	Iris	0.933	0.933	0.987
ET	Custom	Iris	0.967	0.967	0.993
ET	Sklearn	Iris	0.933	0.933	0.990

Table 1: Performance on Iris Dataset

4.1.2 Wine Dataset

Model	Impl	Dataset	Accuracy	F1	AUROC
DT	Custom	Wine	1.000	1.000	1.000
DT	Sklearn	Wine	0.972	0.971	0.976
RF	Custom	Wine	1.000	1.000	1.000
RF	Sklearn	Wine	1.000	1.000	1.000
ET	Custom	Wine	0.972	0.974	1.000
ET	Sklearn	Wine	1.000	1.000	1.000

Table 2: Performance on Wine Dataset

4.2 Hyperparameter Analysis

Figures in this section illustrate the impact of hyperparameters.

4.2.1 Decision Tree

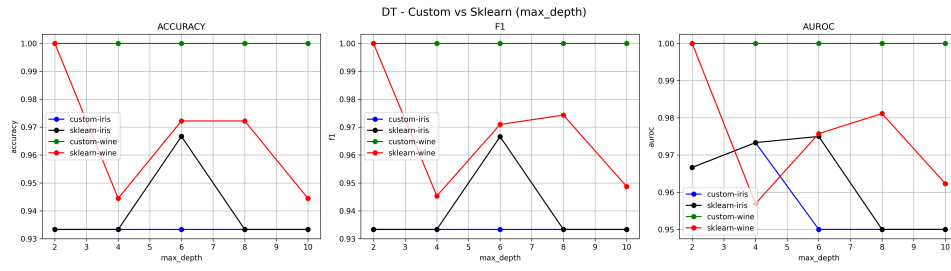


Figure 1: Decision Tree: Effect of Max Depth

4.2.2 Random Forest

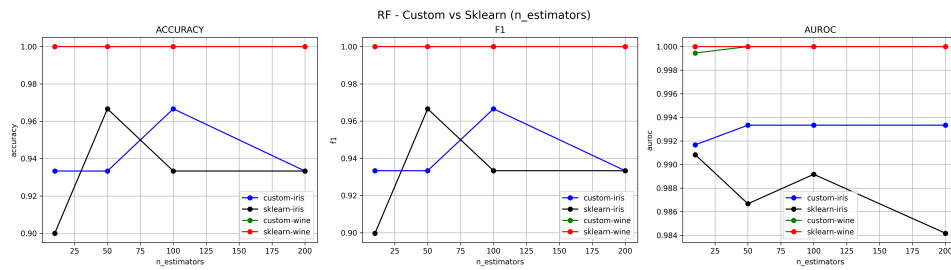


Figure 2: Random Forest: Effect of Number of Trees

4.2.3 Extra Trees

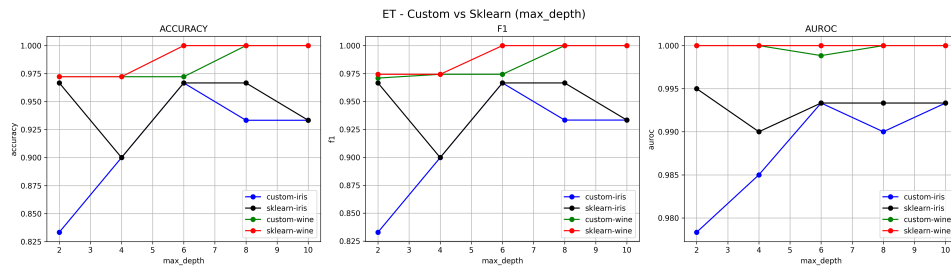


Figure 3: Extra Trees: Effect of Max Depth

4.3 Decision Trees vs Ensemble Methods

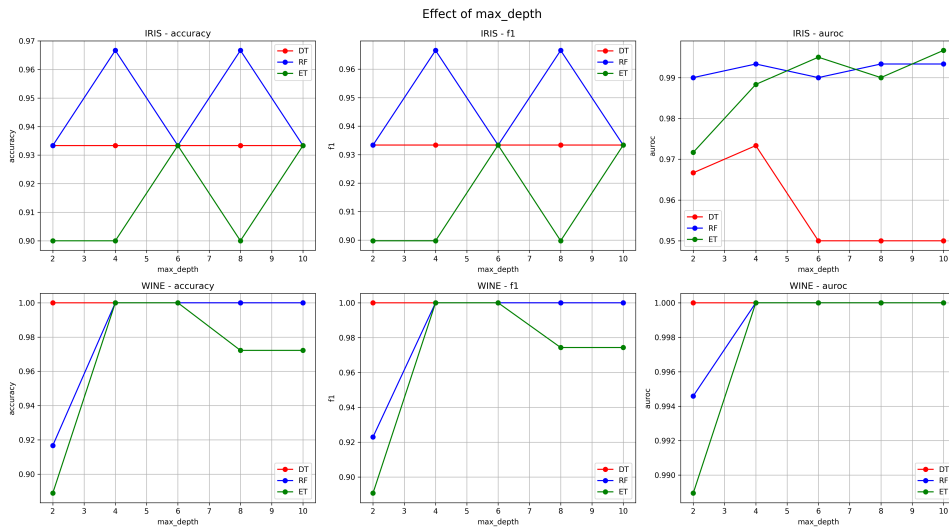


Figure 4: Model Comparison: Max Depth

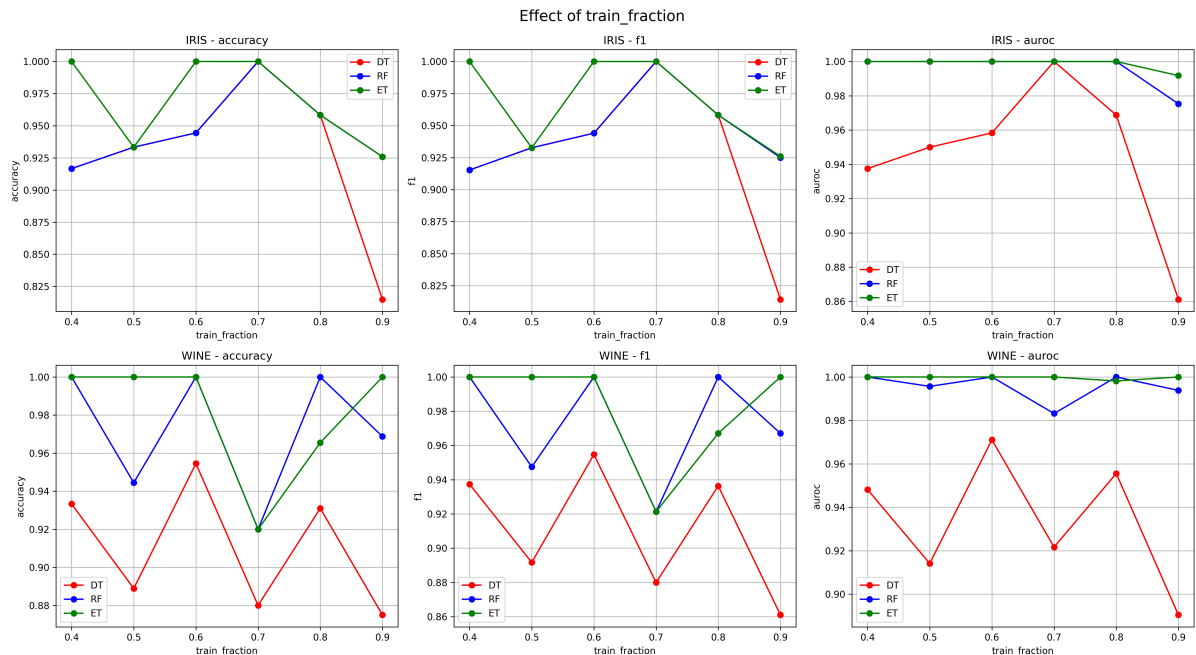


Figure 5: Model Comparison: Training Size

Ensemble models consistently outperform single decision trees, especially on deeper trees and larger datasets, demonstrating reduced variance.

4.4 Confusion Matrix Analysis

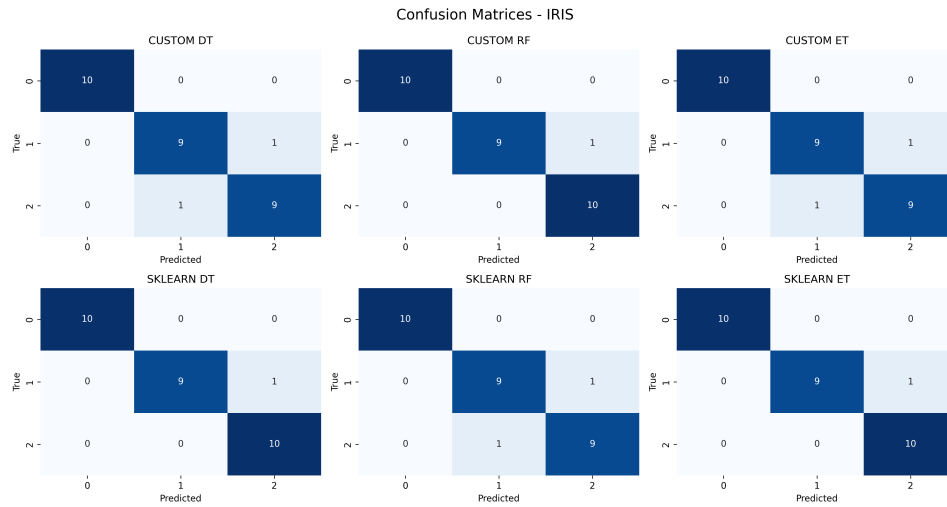


Figure 6: Confusion Matrices on Iris Dataset

These matrices demonstrate improved class-wise prediction stability for ensemble models.

5 Conclusion

This work implemented Decision Trees, Random Forests, and Extra Trees from scratch and evaluated their performance empirically.

The results demonstrate that:

- Ensemble methods significantly reduce overfitting
- Randomization improves generalization
- Custom implementations closely match sklearn
- Bias-variance tradeoffs are clearly observable

Overall, the objectives of the assignment were successfully achieved.